Technical report — `Treuhand Shield`

A privacy proxy that lets Swiss fiduciary offices use Apertus on client files, air-gapped.

- **Track:** `Track 2B — Treuhand Shield: Apertus on client files without exposing client data`
- **Event:** Online
- **Team:** `INNOVA` — Maksym Stepanenko (INNOVA CONSULT LTD, Ottawa)
- **Demo:** https://www.youtube.com/watch?v=lbwxmPPdwZE (83 s)
- **Code:** https://github.com/ctmakc/treuhand-shield (Apache-2.0)

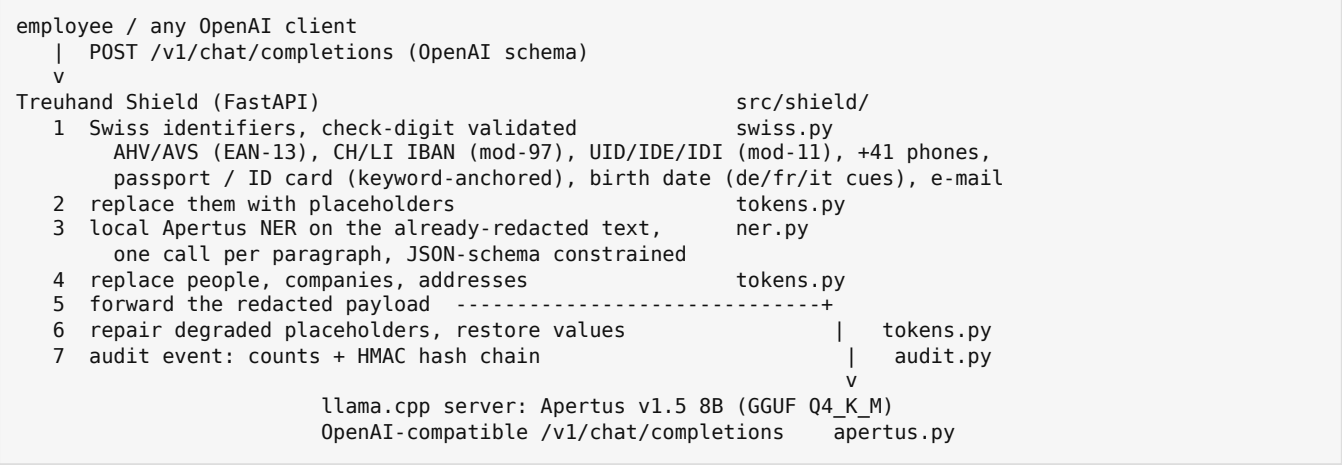
1. Summary

A Swiss fiduciary office (Treuhandbüro) works on documents full of AHV numbers, IBANs, company UIDs, birth dates and home addresses. The staff would gladly hand routine work to an LLM (task lists from a dossier, summaries, first drafts of letters), but client data must not leave the office, and a local model alone leaves no record of what was processed.

Treuhand Shield is an OpenAI-compatible proxy in front of Apertus. Every request is scanned; each sensitive value is replaced by a reversible placeholder such as `[[AHV_3f9a1c]]`; the redacted prompt goes to Apertus; the real values are put back into the answer before the employee sees it; one hash-chained audit event with entity counts (never values) is written per request. Apertus has two roles: it answers, and it is the local detector for names, companies and addresses in German, French and Italian. Structured Swiss identifiers are found by their check digits.

Headline results with Apertus v1.5 8B on one desktop machine, no internet route: on the synthetic Swiss dossier none of the 26 sensitive values reaches the answering model, all placeholders in Apertus' answer are restored, and the audit chain detects an edited event. On 30 generated client notes, the Apertus NER pass cuts the share of sensitive values reaching the model from 42.5 % (identifier patterns alone) to 1.7 %, while task fidelity is 93.3 % through the proxy against 100 % when the raw note goes straight to the model (section 5).

2. Architecture



Design choices:

- **Identifiers first, NER second.** The NER pass runs on text in which every validated identifier is already a placeholder. Even the local model never sees an AHV number or an IBAN, and the shorter text gives the 8B model less to get wrong.
- **One NER call per paragraph.** Our first version made one NER call on the whole raw three-client dossier; the 8B model found 7 of the 9 names, companies and addresses (it missed two of the three client names). With identifiers replaced first and one call per paragraph it found all 9.
- **Placeholders are keyed HMACs** of type and value with a random per-session key, so the same value gets the same placeholder within a session (the model can still reason about "the client" and "her IBAN"), and a placeholder reveals nothing about the value. The mapping lives only in process memory.

- **Two endpoints, one model family.** `LLM_BASE_URL` is the answering model, `NER_BASE_URL` (default: the same) the detector. An office can keep detection on a local Apertus and send redacted prompts to a hosted Apertus in a Swiss cloud; the raw text then never leaves the building, even for NER.

File	What it does
`src/shield/swiss.py`	Swiss identifier patterns and validators `ahv_ean13`, `ch_iban`, `ch_uid`, `ch_phone`
`src/shield/ner.py`	Apertus NER: prompt with one worked example, JSON schema with `maxItems`, salvage of truncated replies, paragraph chunking, label-word filter
`src/shield/tokens.py`	placeholder map, span merge, restore, repair of degraded placeholders, model instruction
`src/shield/audit.py`	HMAC hash-chained audit log (memory or JSON lines)
`src/shield/proxy.py`	`/v1/chat/completions`, `/v1/models`, `/health`
`src/shield/apertus.py`	client for any OpenAI-compatible Apertus server (`LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY`)
`src/demo.py`	end-to-end demo with wire capture, leak checks and an audit tamper test
`src/eval_ch.py`	generated de/fr/it evaluation set and metrics
`docker-compose.yml`, `Makefile`	air-gapped deployment used by `make run`
`tests/test_shield.py`	24 unit tests, no model and no network needed

All of it was written during the hackathon (1–16 October 2026); the git history of this repository starts from the Hack Apertus template.

Target architecture (mandatory)

b) Air-gapped, which also covers **a) On-premise**.

`make run` starts three containers. `fetch-model` is a setup step and the only container with internet access: it downloads the GGUF once and verifies its SHA-256. `llm` (llama.cpp server) and `demo` (proxy plus demo script) sit on a Docker network declared `internal: true`, which has no route to the internet. The demo probes `huggingface.co:443` and `1.1.1.1:443` at start-up and prints the result; in the compose run both are blocked. `make serve` runs the proxy as a service on `127.0.0.1:8000` for the office's own tools; the model container stays on the internal network.

	Build time / setup	Runtime
Container images	`python:3.12-slim`, `ghcr.io/ggml-org/llama.cpp:server-b11561`, `curlimages/curl:8.10.1`, PyPI packages in `requirements.txt`	none
Model weights	one download from Hugging Face (5.06 GB), checksum pinned	read-only volume
Network	Docker Hub, GHCR, PyPI, Hugging Face	internal network only

For **c) Sovereign Swiss cloud**, `make run-remote` runs the same demo against a hosted Apertus endpoint given by `LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY`.

3. Use of Apertus

- **Model:** Apertus v1.5 8B, text-only, GGUF Q4_K_M: `Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF`, a quantisation of `andreasmartin/apertus-v1.5-8b-text` (the text tower of `swiss-ai/Apertus-v1.5-8B`), Apache-2.0, SHA-256 `a037df8d87ff6caacee794ee85f55342f2152e0d359b7389033300c3bee5f299`.
- **How it is used:** inference in two roles: (1) answering model behind the proxy, (2) structured NER for free-text personal data.
- **Where it runs:** local weights, llama.cpp `llama-server`, OpenAI-compatible API, `--jinja` so the chat template embedded in the GGUF is used. Context 8192, one slot.

Answer role. When the redacted prompt contains placeholders, the proxy prepends a system instruction: a token is the value, write it complete with brackets, plus one correct and three wrong examples. A small model follows a formatting rule much better with a concrete wrong example. As a safety net the proxy rewrites degraded forms (`AHV c4857b`, `AHV_c4857b`, `[AHV_c4857b]`) back to the canonical token, only for codes that were present in the request, so the repair can never create a mapping that did not exist.

NER role. System prompt in English, one synthetic German example, temperature 0, and `response_format: json_schema` with an enum of types (PERSON, ORG, ADDRESS, DATE_OF_BIRTH) and `maxItems: 20`. llama.cpp compiles the schema into a grammar, so the reply is always valid JSON of the right shape, and the item cap stops a repetition loop before `max_tokens`. A reply cut off anyway is salvaged item by item. Returned strings are re-anchored in the source text; anything the model invented is discarded. In our first runs Apertus also tagged field labels next to a placeholder (`AHV`, `IBAN`, `Conto`, `Ditta`) as companies; the prompt now lists those labels and a filter drops them.

Other models. None. Development and evaluation used only Apertus; the evaluation metrics are string matches, no LLM judge.

4. Data

All data is synthetic and generated by us; there is no personal data in the repository.

- `data/swiss_dossier.json`: one fiduciary dossier with three clients (German, French and Italian section), 26 labelled sensitive values. Names use the Swiss placeholder names Muster / Exemple / Esempio, e-mails use the reserved `example.org` domain, and every identifier was computed with a valid check digit for an invented body.
- `data/eval_set.jsonl`: 30 client notes produced by `src/eval_ch.py` with seed 7 (10 per language, two templates each), every value labelled with its type.

Licence: CDLA-Permissive-2.0. Size: under 50 KB.

5. Evaluation

Setup. Apertus v1.5 8B Q4_K_M on llama.cpp, AMD Ryzen 5 5600G, NVIDIA RTX 3060 12 GB (shared with other jobs during our runs, so only part of the layers were offloaded). Commands: `python src/demo.py`, `python src/eval_ch.py 30`.

Dossier demo (`src/demo.py`, 26 labelled values, one request through the proxy):

Check	Result
structured identifiers sent to the model in clear (AHV, IBAN, UID, phone, passport, DOB, e-mail)	0 of 17
names, companies, addresses sent in clear (Apertus NER)	0 of 9
placeholders in Apertus' answer left unresolved	0
raw values in the audit log	0
audit chain valid / detects one edited event	yes / yes
end-to-end latency, GPU (all layers offloaded)	21.6 s (NER per paragraph + answer)
end-to-end latency, CPU only (`make run`, Docker, 6 cores)	387.1 s; the whole `make run` incl. image build 7 min 13 s

The transcript of the air-gapped `make run` is in `docs/results/make_run_transcript.txt`.

Generated notes (`src/eval_ch.py`, 30 notes, 294 labelled values). Leak rate = share of labelled values that still appear in the text sent to the model. Task fidelity = share of the three requested values (name, AHV, IBAN) present verbatim in the answer the user receives.

Setup	Leak rate	Task fidelity
Baseline: raw note straight to Apertus (no proxy)	100 % by construction	100 %
Swiss identifier patterns only	42.5 %	n/a
Patterns + Apertus NER (Treuhand Shield)	1.7 %	93.3 %

Details (`docs/results/eval_report.json`, every answer included):

- **Identifier patterns alone** replace every AHV, IBAN, UID, phone, e-mail and passport number (0 leaks in 144 values). They cannot see names (30), companies (30), addresses (60) and five birth dates written without a keyword, as in `Lea Brunner (14.03.1981)`.
- **With the Apertus NER pass** the remaining leaks are 5 address parts; no person name, company or birth date reached the model, and the NER pass added no false positive in 30 notes. Before we added the label filter, Apertus tagged field labels such as `AVS`, `Conto` or `Ditta` as companies (see section 3).
- **Fidelity.** Apertus wrote 80 placeholders in a degraded form (for example `AHV_514251` without brackets); the repair step fixed them and no placeholder was left unresolved in any final answer. The gap to the baseline comes from four French notes that start with `Client : <name>`: the model echoed the label `Client` as the first field and shifted the rest, losing the AHV token in three of them. A previous full run of the same code gave 90.0 %; at temperature 0 the GPU build is not bit-for-bit deterministic. We also tried a longer instruction that tells the model labels are not names; it scored 80.0 % on a full run, so we kept the short one.

Unit tests. `make test`: 24 passed. They cover every Swiss validator with valid and invalid synthetic values, untouched random numbers, keyword-anchored values, NER reply parsing and salvage, label filtering, chunking, placeholder round trip and repair, the audit chain (edit and delete detection), and the proxy route end to end with a fake model.

6. Limitations

- **NER is probabilistic.** Detection of names, companies and addresses depends on the 8B model and varies slightly between runs, even at temperature 0 on GPU. Structured identifiers do not depend on the model.
- **No streaming yet.** The proxy answers `stream=false` requests; repair and restore in a streaming path are next.
- **Passport and identity-card numbers** are matched by shape (letter + 7 digits) next to a keyword; there is no standalone check digit to validate.
- **Phone numbers** without separators in the national format (`0791234567`) are not matched, to avoid redacting arbitrary 10-digit references.
- **Mapping in memory.** Placeholders are per request; a multi-turn chat that should keep the same placeholders across turns needs a session store, which this prototype does not have.
- **Synthetic data only.** We did not test on real client documents, scanned PDFs or OCR output.
- **Quantised community build.** The GGUF is a third-party Q4_K_M conversion of the text tower of the swiss-ai weights. We did not measure the gap to the BF16 weights or to 70B.
- **CPU speed.** Without a GPU the 8B model generates a few tokens per second on a 6-core desktop CPU, so `make run` takes several minutes.

7. Reproducibility

- Hardware used: AMD Ryzen 5 5600G (6 cores), 128 GB RAM, NVIDIA RTX 3060 12 GB. `make run` uses CPU only and needs about 8 GB RAM and 6 GB disk for the weights.
- `make run` (Docker): downloads the weights once, starts llama.cpp on an internal network, runs the demo. Exit code 0 means no structured identifier reached the model and the audit chain behaved as expected. Output also lands in `out/demo_report.json`.
- `make eval`: the 30-note evaluation in the same air-gapped setup (`out/eval_report.json`).
- `make run-remote` with `LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY`: the same demo against a hosted Apertus endpoint.
- `make test`: the unit suite (no model, no network).
- Without Docker: `llama-server -m apertus-v1.5-8b-text-q4_k_m.gguf --jinja --port 8080`, then `pip install -r requirements.txt` and `LLM_BASE_URL=http://localhost:8080/v1 python src/demo.py`.
- Seed 7 for the generated notes. Results in `docs/results/` (demo transcript, demo report, evaluation report with every answer).

8. Next steps

- Apertus 70B through a sovereign Swiss endpoint, compared side by side with the local 8B.
- A small LoRA on Apertus 8B for personal-data spans in Swiss German, French and Italian administrative text, trained on generated data like `eval_set.jsonl`.
- More Swiss identifiers: cantonal tax numbers, health-insurance card numbers, QR-bill references.
- Streaming answers, and a session store so placeholders stay stable across a conversation.
- A pilot with a fiduciary office on consented real documents.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced. The code in this repository is licensed under Apache License 2.0.

References

- Apertus team, *Apertus: Democratizing Open and Compliant LLMs*, arXiv:2509.14233 (2025).
- swiss-ai/Apertus-v1.5-8B, https://huggingface.co/swiss-ai/Apertus-v1.5-8B
- llama.cpp, https://github.com/ggml-org/llama.cpp
- ZAS / CdC, AHV number format (756.XXXX.XXXX.XC, EAN-13 check digit).
- Federal Statistical Office, UID register: CHE-XXX.XXX.XXX with mod-11 check digit.
- ISO 13616 (IBAN), mod-97 check.